

Mac MOE Model Inference

is there a inference engine out there that specific design for mac that load moe model on ssd, and only load the expert weight to ram and process token with gpu or ANE, unload and repeat the same process next token

RouteLLM

Yes — **Flash-MoE** is very close to exactly what you describe, and is purpose-built for Apple Silicon Macs.

It is a C/Metal inference engine that:

- Keeps the model's non-expert/shared weights resident.
- Routes each token at each MoE layer.
- Reads only the selected experts from the Mac's NVMe SSD on demand.
- Runs the expert computation through custom **Metal GPU** kernels.
- Relies on macOS's page cache to retain recently used expert pages rather than explicitly maintaining its own GPU/RAM expert cache.

Its published implementation targets an M3 Max MacBook Pro and streams a 397B-parameter MoE model from SSD. The important nuance: expert selection occurs **per layer, per token**, not just once per generated token. Therefore the actual sequence is effectively:

GPU backbone/router → select experts → SSD read → GPU expert compute repeated for each MoE layer. It reported about 4.4 tokens/sec for a 209 GB 4-bit model on its stated hardware, but treat that as the project's reported benchmark rather than a general guarantee. See its [architecture and benchmarks](#).

Another option is **moe-stream**, a Rust/Metal engine for Apple Silicon. It has an explicit **SSD Streaming** mode and uses mmap / NVMe-backed model access. But according to its own design documentation, this mode uses Metal primarily for selected operations while much of the streaming-mode pipeline—including MoE expert computation—is CPU-side. So it is less aligned with your “load selected experts and execute them on GPU” requirement.

ANE caveat

I'm not aware of a production-quality Mac inference engine that uses the **Apple Neural Engine** for this dynamic, per-expert SSD streaming design. The ANE is generally a poor fit for arbitrary per-layer, dynamically selected weight loading: this workflow needs flexible custom kernels and memory management, which is why these engines use **Metal on the Apple GPU** instead.

Best match

Use **Flash-MoE** if you want the closest existing implementation of:

SSD-resident MoE experts → demand-load selected experts → GPU compute → repeat.

It also illustrates why you usually should **not eagerly unload every expert** after use: macOS's unified-memory page cache can preserve hot expert pages automatically, avoiding repeat SSD reads when routing reuses them.

between <https://github.com/danveloper/flash-moe/tree/main> ,
<https://github.com/haihengh/turbo-fieldfare> and
<https://github.com/haihengh/omlx>, help me to come up with an
engine design specifically for apple hardware to run qwen 3.6 35b
a3b model or the variant model like opus distilled and uncensor
models, make it highly optimized to run on apple hardware with
limited ram, target below using 4g of active ram

RouteLLM

Recommended design: AeroMoE — SSD-streamed Metal runtime for Apple Silicon

For a **Qwen 3.6 35B-A3B-class MoE** and derived fine-tunes, I would build a **model-family-aware engine**, not a general-purpose MLX wrapper. The closest base architecture is **TurboFieldfare**, combined with Flash-MoE's low-level Metal and I/O lessons, while borrowing oMLX only at the serving/KV-cache layer.

Your hard requirement—**under 4 GB active unified memory**—is feasible only if that means **engine-owned active memory** excluding macOS file-cache pages. It is not feasible to hold all model weights resident in 4 GB. The engine must keep the dense/shared path compact and SSD-stream routed experts per layer.

The exact resident-memory budget depends on the actual checkpoint config: layers, hidden size, expert count, top-k, shared expert structure, attention type, and KV dimensions. Do not hard-code it around the marketing label “35B-A3B”; inspect each checkpoint’s configuration and tensor shapes at conversion time.

```
<svg width="100%" viewBox="0 0 680 400" xmlns="http://www.w3.org/2000/svg" role="img"
aria-label="AeroMoE Apple Silicon streaming architecture">
  <defs>
    <marker id="a" markerWidth="8" markerHeight="8" refX="7" refY="4" orient="auto"><path
d="M0,0 L8,4 L0,8 Z" fill="var(--color-text-secondary)"/></marker>
  </defs>
  <style>
    text{font-family:var(--font-sans);fill:var(--color-text-primary)}.th{font-size:14px;font-
weight:500}.ts{font-size:12px;fill:var(--color-text-secondary)}.arr{stroke:var(--color-text-
secondary);stroke-width:1.2;fill:none;marker-end:url(#a)}.box{stroke-width:.6}
  </style>
  <g class="c-gray">
    <rect class="box" x="18" y="35" width="128" height="305" rx="12"/>
    <text class="th" x="82" y="62" text-anchor="middle">NVMe SSD</text>
    <text class="ts" x="82" y="81" text-anchor="middle">packed expert shards</text>
    <rect x="35" y="108" width="94" height="38" rx="5" fill="var(--color-background-tertiary)"
stroke="var(--color-border-secondary)" stroke-width=".5"/>
    <text class="ts" x="82" y="127" text-anchor="middle" dominant-baseline="central">L00
experts</text>
    <rect x="35" y="160" width="94" height="38" rx="5" fill="var(--color-background-tertiary)"
stroke="var(--color-border-secondary)" stroke-width=".5"/>
    <text class="ts" x="82" y="179" text-anchor="middle" dominant-baseline="central">L01
experts</text>
    <rect x="35" y="212" width="94" height="38" rx="5" fill="var(--color-background-tertiary)"
stroke="var(--color-border-secondary)" stroke-width=".5"/>
    <text class="ts" x="82" y="231" text-anchor="middle" dominant-baseline="central">...
layer shards</text>
  </g>
  <path class="arr" d="M146 185 L198 185"/>
  <g class="c-amber">
    <rect class="box" x="199" y="122" width="145" height="126" rx="12"/>
    <text class="th" x="271" y="150" text-anchor="middle">I/O + cache planner</text>
    <text class="ts" x="271" y="171" text-anchor="middle">bounded parallel pread</text>
    <text class="ts" x="271" y="190" text-anchor="middle">8–16 expert slots</text>
    <text class="ts" x="271" y="209" text-anchor="middle">LFU / recency admission</text>
  </g>
  <path class="arr" d="M344 185 L393 185"/>
  <g class="c-purple">
    <rect class="box" x="394" y="35" width="267" height="305" rx="16"/>
    <text class="th" x="527" y="62" text-anchor="middle">Apple unified memory + Metal
```

```

GPU</text>
  <rect x="414" y="88" width="227" height="52" rx="7" fill="var(--color-background-
secondary)" stroke="var(--color-border-secondary)" stroke-width=".5"/>
  <text class="th" x="527" y="107" text-anchor="middle" dominant-
baseline="central">Resident backbone</text>
  <text class="ts" x="527" y="126" text-anchor="middle" dominant-
baseline="central">embedding · attention · router · shared path</text>
  <rect x="414" y="159" width="227" height="52" rx="7" fill="var(--color-background-
secondary)" stroke="var(--color-border-secondary)" stroke-width=".5"/>
  <text class="th" x="527" y="178" text-anchor="middle" dominant-baseline="central">Metal
expert slots</text>
  <text class="ts" x="527" y="197" text-anchor="middle" dominant-baseline="central">direct
Q4 matvec + fused SwiGLU</text>
  <rect x="414" y="230" width="227" height="52" rx="7" fill="var(--color-background-
secondary)" stroke="var(--color-border-secondary)" stroke-width=".5"/>
  <text class="th" x="527" y="249" text-anchor="middle" dominant-
baseline="central">Bounded KV cache</text>
  <text class="ts" x="527" y="268" text-anchor="middle" dominant-baseline="central">paged
· quantized · sliding window first</text>
  <text class="ts" x="527" y="316" text-anchor="middle">one token: backbone → route →
fetch → MoE → next layer</text>
</g>
</svg>

```

What to reuse from each project

Source	Reuse	Avoid / change
Flash-MoE	Hand-written Metal, direct quantized matvec, fused operations, pread()-based SSD reads, treating Apple unified-memory bandwidth as the primary constraint	Its architecture is tailored to a very different model family; do not use its tensor layout or routing assumptions blindly. Its “trust macOS page cache” approach is less safe on an 8 GB machine.
TurboFieldfare	The most relevant blueprint: strict resident-memory accounting, dedicated model format, per-layer expert streaming, bounded cache, async overlap of shared-expert GPU work with SSD reads, chunked prefill	Its Gemma-specific graph, top-8 behavior, tensor names, and .gturbo format must be replaced with a Qwen-family converter and runtime plan.
oMLX	Server lifecycle, OpenAI-compatible interface, paged KV-cache ideas, request scheduling, model management patterns	Do not put Python/MLX in the hot decode loop for this target. It is a robust serving layer, not a sub-4 GB expert-streaming execution core.

Memory contract

Target a **3.6–3.9 GB engine allocation ceiling**, leaving a safety margin for macOS, UI, Metal driver allocation, and file cache.

Region	Target allocation	Notes
Quantized resident backbone	1.6–2.1 GB	Embedding, output head, attention projections, norms, routers, shared experts, plus any non-MoE FFNs. Must be Q4 or mixed Q4/Q5.
KV cache	0.6–1.0 GB	Context-dependent. Use paged allocation and quantization; do not promise full long context on 8 GB machines.
Metal activation/scratch arena	192–320 MB	Fixed-size, reusable; no allocation within decoding.
Routed-expert slots	512–896 MB	A small cache of complete expert tensors, sized using actual converted expert sizes .
CPU scheduler, tokenizer, metadata	100–180 MB	Keep router output small; no large CPU tensor copies.
Total	≤ 3.9 GB	Enforced by a runtime allocator and admission control.

The **main feasibility gate** is the non-expert core. If the Qwen-derived model's required resident backbone cannot be packed below about 2.1 GB without unacceptable accuracy degradation, then a genuine 4 GB runtime budget is not achievable. Do not silently spill backbone tensors to SSD: that would cause an SSD fetch in every layer regardless of routing and collapse decode speed.

Model storage format: .aeromoe

Do not stream generic safetensors or ordinary MLX shards during token decode. Build a conversion pipeline that emits an inference-only format:

```
model.aeromoe/
manifest.json    model shape, context/KV layout, quant schemes
tokenizer.*      copied tokenizer assets
core.bin         packed resident tensors, 4 KB-aligned
experts/
  layer_000.bin   all routed experts of layer 0
  layer_001.bin
  ...
index.bin        offset, byte length, quant metadata per expert
```

Design rules

- **One independently readable, contiguous slice per expert.**
 - Each slice includes all matrices needed by that expert: gate/up projection and down projection.
 - Alignment should favor direct I/O and Metal buffer binding; use a converter-defined, fixed alignment such as 64 KB.
- **Separate files per transformer layer** are preferable to a huge interleaved file. The decoder knows which layer it is executing, and the I/O planner can read a selected expert as one contiguous range.
- **Pack directly for Metal.** The SSD representation must be the GPU-consumable layout—not an intermediate format that requires unpacking or dequantizing into a larger buffer.
- **Mixed quantization:**
 - Router: 8-bit.
 - Norms, RoPE constants, and numerically sensitive small tensors: FP16 or FP32 where necessary.
 - Dense backbone: Q4 with Q5/Q6 selectively for quality-sensitive tensors.
 - Routed experts: Q4 first; Q5 is an optional quality profile, not the low-RAM default.
- The converter must validate architecture and tensor coverage before writing output. “Opus distilled” or uncensored variants may alter tokenizer/configuration, but should only be accepted if their actual graph matches a supported Qwen adapter.

Decode execution plan

Per generated token, per transformer layer:

1. **Backbone Metal command buffer A**
 - RMSNorm.

- Q/K/V and output projections.
- RoPE and decode attention against the paged KV cache.
- Router logits.
- Router top-k selection on GPU if practical; otherwise copy only the small expert-ID/weight result to CPU.

2.

CPU planner

- Looks up the selected expert slices in index.bin.
- Checks the bounded Metal-visible expert-slot cache.
- Submits cache misses as bounded, parallel pread() operations.
- Performs no weight transformations and no full-tensor copies.

3.

Overlap only useful work

- While SSD reads run, execute the **shared expert** or any independent layer operations in Metal command buffer B.
- Do **not** aggressively prefetch predicted experts. Flash-MoE's published experiments found that speculative routing/prefetch hurt performance due to poor hit rate and unified-memory contention.

4.

Expert Metal command buffer C

- Bind resident cache hits and freshly filled slots.
- Quantized GEMV for gate/up.
- Fuse SiLU and element-wise gating.
- Quantized down projection.
- Apply routing weights, combine output, residual add, and next RMSNorm in as few passes as possible.

5.

Eviction

- Evict only after a slot is needed.
- Use a small adaptive policy: protected “used this token/layer” set + frequency score + recency tiebreaker.
- On an 8 GB Mac, issue F_NOCACHE or equivalent only for data unlikely to be reused; otherwise macOS may fill available memory with expert pages and create memory pressure.

The key difference from the simplistic “load then unload every token” design is that loading is still demand-driven, but recently used experts remain in a **strictly capped cache**. This is essential because routing repeats enough that blind unloads waste SSD bandwidth.

Apple-specific choices

GPU: Metal is the primary execution device

Use **Metal**, not ANE, for the decode path:

- You need custom Q4/Q5 matrix-vector kernels, dynamic expert binding, command-buffer control, and predictable buffer ownership.
- The ANE is not a good fit for frequent dynamic expert-swapping or low-level MoE routing.
- Apple Silicon has unified memory, so there is no PCIe transfer between CPU and GPU. The bottleneck is **shared DRAM bandwidth and SSD DMA contention**, not GPU-to-CPU copying.

I/O

- Use `pread()` on a dedicated I/O queue with a small number of in-flight reads.
- Start with **2–4 parallel reads**, then autotune by machine class. More concurrency can increase latency through memory-controller contention.

- Allocate fixed, aligned, Metal-visible buffers for expert slots. Reads should fill those buffers directly if the API path allows it; otherwise copy only once.
- Benchmark cold SSD, warm OS cache, and memory-pressure cases separately. Warm-cache throughput is useful but should not be advertised as sustained cold-cache throughput.

CPU

- Use Swift for packaging/UI/service orchestration if desired, but put the runtime core in **C++/Objective-C++ plus Metal** or Swift with a thin C/Obj-C Metal layer.
- Use GCD / Swift concurrency for I/O coordination, but avoid task allocation in inner decode loops.
- Pin no threads manually until profiling proves a benefit; Apple's scheduler generally does better with QoS and bounded work queues.

Prefill and KV strategy

Decode is the memory-critical mode. Prefill needs different behavior:

- Process prefill in **small chunks**—start with 32 tokens, autotune up to 128 only if memory permits.
- Deduplicate selected experts across all rows in a chunk: a fetched expert serves every routed row that needs it.
- Use a paged KV layout with a hard token budget.
- For a 4 GB active-memory profile:
 - Default to a conservative context cap.
 - Prefer **sliding-window attention** where the model supports it.
 - Quantize older KV blocks before considering SSD KV offload.
 - SSD KV is a latency recovery mechanism for resumed prefixes, not part of the critical decode loop.

oMLX's tiered KV-cache approach is useful for resumed conversation prefixes, but expert SSD streaming and SSD KV restoration must never compete unrestrictedly. Give expert I/O priority during generation.

Build order

1.

Architecture probe + converter

- Parse the actual model config and weights.
- Emit manifest, core.bin, layer-expert shards, and index.
- Add an offline reference test comparing individual layer outputs to the source implementation.

2.

Resident-only Metal decoder

- Load a small compatible Qwen-family MoE entirely into memory.
- Validate tokenizer, RoPE, attention, routing, and exact expert combination before adding SSD behavior.

3.

One-layer expert-streaming prototype

- Replace one MoE layer with SSD pread() and 2–4 fixed expert slots.
- Verify numerics and profile real cold-read latency.

4.

Full streaming and capped memory allocator

- Add strict allocation accounting.
- Fail fast or reduce KV capacity when the 3.9 GB ceiling would be exceeded—never let macOS swap.

5.

Prefill, server, and fine-tune adapters

- Add chunked prefill.
- Then expose a local OpenAI-compatible endpoint.
- Support additional distills/fine-tunes through explicit architecture adapters and conversion validation.

What performance to expect

I would **not promise a token/sec figure** without the target Mac model and a converted checkpoint. It will be driven by:

- Apple GPU and unified-memory bandwidth,
- internal SSD speed and free space,
- actual expert size and top-k,
- resident-core size,
- context/KV-cache size,
- cache hit rate,
- quantization quality profile,
- whether the model's dense/shared path fits the resident budget.

The appropriate success criterion for version one is:

correct output matching a reference runtime within an agreed tolerance, no system swap, active engine allocation below 4 GB, and stable cold-cache decoding.

TurboFieldfare demonstrates the closest relevant proof point: it reports a 26B-A4B model with a roughly 2 GB weight-and-KV budget by keeping a 1.35 GB shared core resident and streaming selected experts from SSD. TurboFieldfare's design is

therefore the best structural starting point; Flash-MoE supplies the stronger low-level kernel and SSD-streaming techniques for large MoEs.

review the design, and help me to come up with key inference engine code, make it as detail as possible, output the document in md format

Claude Fable 5

AeroMoE — Design Review & Core Inference Engine Code

Part 1: Design Review

What holds up

Decision	Verdict
Metal-only, no ANE	✔ Correct — dynamic expert binding is impossible to do cleanly on ANE
Layer-sharded expert files, contiguous per-expert slices	✔ Correct — one pread() per miss
Bounded slot cache instead of load/unload per token	✔ Correct — routing locality makes blind unloads waste SSD bandwidth
No speculative prefetch	✔ Correct for v1 — revisit only with measured hit rates
Overlap shared-expert GPU work with SSD reads	✔ Correct, this is the single biggest latency hiding win

Weaknesses to fix before coding

- Router → CPU sync point.** The GPU must finish router logits before the CPU can plan I/O. That's a per-layer pipeline stall. Fix: split each layer into two command buffers and read back only top_k × (uint16 id, half weight) via a shared-storage-mode buffer — never a full logits copy. On decode (1 token), do top-k on CPU from the raw logits vector (small, e.g. 128 floats); it's cheaper than a GPU sort + readback.
- Slot fragmentation.** Experts across layers may differ in size. Fix: all slots sized to max_expert_bytes from the manifest. Wastes a few MB, eliminates an allocator.

3. **Eviction during in-flight GPU reads.** A slot must not be evicted while a command buffer still references it. Fix: per-slot refcount pinned until command-buffer completion handler fires.
 4. **pread into Metal buffers.** Use `MTLStorageModeShared` buffers and `pread()` directly into `buffer.contents` — zero-copy on unified memory. Open the expert files with `F_NOCACHE` **off** by default; only enable it under memory pressure (watch `dispatch_source` memory-pressure events).
 5. **Budget enforcement is missing a mechanism.** Add a central `MemoryLedger` that every allocation goes through; fail construction if the plan exceeds the ceiling instead of discovering it at runtime.
-

Part 2: Core Engine Code

Language: C++17 / Objective-C++ (.mm) + Metal Shading Language. Illustrative but structurally complete — tensor shapes come from `manifest.json` at load time, never hard-coded.

2.1 Manifest & expert index

```
// aeromoe_format.h
#pragma once
#include <cstdint>
#include <vector>
#include <string>

namespace aeromoe {

struct QuantSpec {          // per-tensor-group quantization
    enum Kind : uint8_t { Q4_BLOCK32, Q5_BLOCK32, Q8, F16, F32 };
    Kind kind;
    uint32_t block_size;    // elements per quant block (e.g. 32)
};

struct ModelManifest {
    uint32_t n_layers, d_model, n_heads, n_kv_heads, head_dim;
    uint32_t n_experts, top_k;
    uint32_t d_ff_expert, d_ff_shared;    // shared-expert width; 0 if none
    uint32_t vocab_size, max_context;
};
```

```

float rope_theta;
QuantSpec backbone_q, expert_q, router_q;
uint64_t max_expert_bytes;      // sizing for uniform slots
// parsed from manifest.json; validated against tensor coverage
};

// index.bin: one record per (layer, expert), contiguous, mmap-friendly
struct ExpertRecord {
    uint64_t offset;      // into experts/layer_XXX.bin, 64KB aligned
    uint32_t length;      // total bytes: gate+up+down, packed for Metal
    uint32_t gate_up_bytes; // split point inside the slice
};

struct ExpertIndex {
    std::vector<ExpertRecord> records;    // n_layers * n_experts
    const ExpertRecord& at(uint32_t layer, uint32_t expert) const {
        return records[layer * n_experts_ + expert];
    }
    uint32_t n_experts_;
};

} // namespace aeromoe

```

2.2 Memory ledger (the 4 GB contract)

```

// memory_ledger.h
#pragma once
#include <atomic>
#include <cstdint>
#include <stdexcept>

namespace aeromoe {

class MemoryLedger {
public:
    explicit MemoryLedger(uint64_t ceiling_bytes) : ceiling_(ceiling_bytes) {}

    // All engine allocations MUST reserve here first.
    void reserve(uint64_t bytes, const char* tag) {
        uint64_t cur = used_.fetch_add(bytes) + bytes;
        if (cur > ceiling_) {
            used_.fetch_sub(bytes);
            throw std::runtime_error(
                std::string("AeroMoE budget exceeded reserving ") + tag);
        }
    }
};
}

```

```

    }
}
void release(uint64_t bytes) { used_.fetch_sub(bytes); }
uint64_t used() const { return used_.load(); }
uint64_t ceiling() const { return ceiling_; }
private:
    std::atomic<uint64_t> used_{0};
    uint64_t ceiling_;
};

} // namespace aeromoe

```

Construction-time plan check (fail fast, never swap):

```

// Called once during engine init, before any Metal allocation.
void validate_budget(const ModelManifest& m, const EngineConfig& cfg,
    MemoryLedger& ledger) {
    ledger.reserve(cfg.backbone_bytes,    "backbone");    // ~1.6–2.1 GB
    ledger.reserve(cfg.kv_budget_bytes,    "kv_cache");    // ~0.6–1.0 GB
    ledger.reserve(cfg.n_slots * m.max_expert_bytes, "expert_slots");
    ledger.reserve(cfg.scratch_bytes,      "metal_scratch"); // fixed arena
    // If we get here, the plan fits under the ceiling (e.g. 3.9 GB).
}

```

2.3 Expert slot cache (bounded, refcounted, LFU+recency)

```

// expert_cache.h
#pragma once
#import <Metal/Metal.h>
#include <unordered_map>
#include <vector>
#include <mutex>

namespace aeromoe {

struct SlotKey {
    uint32_t layer, expert;
    bool operator==(const SlotKey& o) const {
        return layer == o.layer && expert == o.expert;
    }
};

struct SlotKeyHash {

```

```

size_t operator()(const SlotKey& k) const {
    return (size_t(k.layer) << 32) ^ k.expert;
}
};

struct Slot {
    id<MTLBuffer> buffer;      // MTLStorageModeShared, max_expert_bytes
    SlotKey key{UINT32_MAX, UINT32_MAX};
    std::atomic<int> refcount{0}; // pinned while a cmd buffer uses it
    uint32_t freq = 0;          // decayed frequency score
    uint64_t last_use_token = 0;
    bool valid = false;
};

class ExpertCache {
public:
    ExpertCache(id<MTLDevice> dev, uint32_t n_slots, uint64_t slot_bytes,
                MemoryLedger& ledger) : ledger_(ledger) {
        // slot memory was already reserved by validate_budget()
        slots_.resize(n_slots);
        for (auto& s : slots_)
            s.buffer = [dev newBufferWithLength:slot_bytes
                        options:MTLResourceStorageModeShared];
    }

    // Returns slot if resident (pins it), else nullptr.
    Slot* lookup_and_pin(SlotKey key, uint64_t token_idx) {
        std::lock_guard<std::mutex> g(mu_);
        auto it = map_.find(key);
        if (it == map_.end()) return nullptr;
        Slot* s = &slots_[it->second];
        s->refcount.fetch_add(1);
        s->freq++; s->last_use_token = token_idx;
        return s;
    }

    // Picks a victim: unpinned, lowest (freq, last_use). Caller fills it.
    Slot* acquire_for_fill(SlotKey key, uint64_t token_idx) {
        std::lock_guard<std::mutex> g(mu_);
        Slot* victim = nullptr;
        for (auto& s : slots_) {
            if (s.refcount.load() != 0) continue; // pinned: skip
            if (!s.valid) { victim = &s; break; } // free slot
            if (!victim || std::tie(s.freq, s.last_use_token) <
                std::tie(victim->freq, victim->last_use_token))
                victim = &s;
        }
    }
};

```



```

    }
    if (!victim) return nullptr; // all pinned → caller stalls one read
    if (victim->valid) map_.erase(victim->key);
    victim->key = key; victim->valid = false; // valid after fill
    victim->freq = 1; victim->last_use_token = token_idx;
    victim->refcount.fetch_add(1); // pin for fill+use
    return victim;
}

void publish(Slot* s) { // after pread completes
    std::lock_guard<std::mutex> g(mu_);
    s->valid = true;
    map_[s->key] = uint32_t(s - slots_.data());
}
void unpin(Slot* s) { s->refcount.fetch_sub(1); }

void decay() { // call every N tokens
    std::lock_guard<std::mutex> g(mu_);
    for (auto& s : slots_) s.freq >>= 1;
}
private:
    std::vector<Slot> slots_;
    std::unordered_map<SlotKey, uint32_t, SlotKeyHash> map_;
    std::mutex mu_;
    MemoryLedger& ledger_;
};

} // namespace aeromoe

```

2.4 I/O planner — bounded parallel pread into Metal buffers

```

// io_planner.mm
#import <Metal/Metal.h>
#include <dispatch/dispatch.h>
#include <fcntl.h>
#include <unistd.h>
#include "expert_cache.h"
#include "aeromoe_format.h"

namespace aeromoe {

class IOPlanner {
public:
    IOPlanner(const ExpertIndex& idx, std::vector<int> layer_fds,

```

```

    ExpertCache& cache, uint32_t max_inflight = 3)
: idx_(idx), fds_(std::move(layer_fds)), cache_(cache) {
    q_ = dispatch_queue_create("aeromoe.io", DISPATCH_QUEUE_CONCURRENT);
    sem_ = dispatch_semaphore_create(max_inflight); // autotuned 2-4
    grp_ = dispatch_group_create();
}

// For each selected expert: hit → pin; miss → async pread into a slot.
// Returns pinned slots in expert order; blocks until all fills complete.
std::vector<Slot*> fetch(uint32_t layer,
                        const std::vector<uint32_t>& experts,
                        uint64_t token_idx) {
    std::vector<Slot*> out(experts.size(), nullptr);
    for (size_t i = 0; i < experts.size(); ++i) {
        SlotKey key{layer, experts[i]};
        if (Slot* hit = cache_.lookup_and_pin(key, token_idx)) {
            out[i] = hit;
            continue;
        }
        Slot* slot = nullptr;
        while (!(slot = cache_.acquire_for_fill(key, token_idx)))
            dispatch_group_wait(grp_, DISPATCH_TIME_FOREVER); // rare
        out[i] = slot;
        const ExpertRecord& rec = idx_.at(layer, experts[i]);
        int fd = fds_[layer];
        dispatch_semaphore_wait(sem_, DISPATCH_TIME_FOREVER);
        dispatch_group_async(grp_, q_, ^{
            // Zero-copy on unified memory: read straight into the
            // MTLBuffer's shared-storage contents.
            void* dst = slot->buffer.contents;
            ssize_t n = pread(fd, dst, rec.length, off_t(rec.offset));
            if (n != (ssize_t)rec.length) abort(); // TODO: error path
            cache_.publish(slot);
            dispatch_semaphore_signal(sem_);
        });
    }
    dispatch_group_wait(grp_, DISPATCH_TIME_FOREVER);
    return out; // every slot pinned; unpin in cmd-buffer completion
}

private:
    const ExpertIndex& idx_;
    std::vector<int> fds_; // one fd per layer file
    ExpertCache& cache_;
    dispatch_queue_t q_;
    dispatch_semaphore_t sem_;
    dispatch_group_t grp_;
};

```

```
} // namespace aeromoe
```

2.5 Metal kernels — Q4 GEMV + fused SwiGLU + weighted combine

```
// expert_kernels.metal
#include <metal_stdlib>
using namespace metal;

// Q4 block layout (block_size = 32): [half scale][half min][16 packed bytes]
struct Q4Block { half scale; half zmin; uchar q[16]; };

inline float dequant_dot(device const Q4Block* row_blocks,
                        device const half* x, uint n_blocks) {
    float acc = 0.0f;
    for (uint b = 0; b < n_blocks; ++b) {
        device const Q4Block& blk = row_blocks[b];
        float s = float(blk.scale), m = float(blk.zmin);
        device const half* xb = x + b * 32;
        for (uint i = 0; i < 16; ++i) {
            uchar p = blk.q[i];
            acc += (s * float(p & 0xF) + m) * float(xb[2*i]);
            acc += (s * float(p >> 4) + m) * float(xb[2*i+1]);
        }
    }
    return acc;
}

// One threadgroup per output row group; simdgroup reduction elided for brevity.
// Fused: y_ff[j] = silu(gate_j · x) * (up_j · x)
kernel void expert_gateup_silu(
    device const Q4Block* gate_w  [[buffer(0)]], // [d_ff][n_blocks]
    device const Q4Block* up_w    [[buffer(1)]],
    device const half* x          [[buffer(2)]], // [d_model]
    device half* y_ff             [[buffer(3)]], // [d_ff]
    constant uint& d_model [[buffer(4)]],
    constant uint& d_ff      [[buffer(5)]],
    uint j [[thread_position_in_grid]])
{
    if (j >= d_ff) return;
    uint nb = d_model / 32;
    float g = dequant_dot(gate_w + j * nb, x, nb);
    float u = dequant_dot(up_w + j * nb, x, nb);
    float silu = g / (1.0f + exp(-g));
```

```

    y_ff[j] = half(silu * u);
}

// down projection + weighted accumulate into the shared MoE output.
// Called once per selected expert; routing weight applied here so no
// separate combine pass is needed.
kernel void expert_down_accum(
    device const Q4Block* down_w  [[buffer(0)]], // [d_model][ff_blocks]
    device const half*   y_ff     [[buffer(1)]],
    device atomic_float* moe_out  [[buffer(2)]], // [d_model], fp32 accum
    constant uint&       d_model  [[buffer(3)]],
    constant uint&       d_ff     [[buffer(4)]],
    constant float&      route_w  [[buffer(5)]],
    uint i [[thread_position_in_grid]])
{
    if (i >= d_model) return;
    uint nb = d_ff / 32;
    float v = dequant_dot(down_w + i * nb, y_ff, nb);
    atomic_fetch_add_explicit(&moe_out[i], route_w * v,
                             memory_order_relaxed);
}

```

Production note: replace the scalar inner loop with simdgroup cooperative loads and 4-block unrolling; the structure (fused SiLU, weight-applied accumulate, fp32 accumulator) is the part that matters.

2.6 The per-layer decode loop with I/O overlap

```

// decode_layer.mm — one MoE layer for one decode token
#include "io_planner.mm"

void DecodeLayer::run(uint32_t layer, id<MTLCommandQueue> queue,
    LayerWeights& W, TokenState& st, IOPlanner& io,
    ExpertCache& cache) {
    // ---- Command buffer A: attention + router (backbone, resident) ----
    id<MTLCommandBuffer> cbA = [queue commandBuffer];
    encode_rmsnorm(cbA, st.x, W.attn_norm);
    encode_attention(cbA, st, W);           // QKV, RoPE, paged-KV decode attn
    encode_rmsnorm(cbA, st.x, W.ffn_norm);
    encode_router_logits(cbA, st.x, W.router, st.router_logits); // shared buf
    [cbA commit];
}

```

```

[cbA waitUntilCompleted];          // unavoidable sync: need routing

// ---- CPU: top-k from tiny logits vector (n_experts floats) ----
auto [ids, weights] = topk_cpu(st.router_logits, manifest_.top_k);

// ---- Overlap: kick SSD reads, run shared expert meanwhile ----
id<MTLCommandBuffer> cbB = [queue commandBuffer];
if (W.has_shared_expert)
    encode_shared_expert(cbB, st.x, W.shared, st.moe_out /*fp32 accum*/);
[cbB commit];                      // runs while pread() proceeds

std::vector<Slot*> slots = io.fetch(layer, ids, st.token_idx); // blocks

// ---- Command buffer C: routed experts ----
id<MTLCommandBuffer> cbC = [queue commandBuffer];
for (size_t i = 0; i < slots.size(); ++i) {
    const ExpertRecord& rec = index_.at(layer, ids[i]);
    // gate/up live at offset 0, down at gate_up_bytes, inside one slot
    encode_expert_gateup_silu(cbC, slots[i]->buffer, 0, st.x, st.y_ff);
    encode_expert_down_accum (cbC, slots[i]->buffer, rec.gate_up_bytes,
                             st.y_ff, st.moe_out, weights[i]);
}
encode_residual_add(cbC, st.x, st.moe_out); // fp32 → fp16 + residual

// Unpin slots only after the GPU is done touching them.
[cbC addCompletedHandler:^(id<MTLCommandBuffer>) {
    for (Slot* s : slots) cache.unpin(s);
}];
[cbC commit];
[cbC waitUntilCompleted]; // v1: sync per layer; v2: pipeline layers
}

```

2.7 Top-level token loop

```

int32_t Engine::decode_one_token(TokenState& st) {
    st.x = embed(st.token_id);          // resident embedding
    for (uint32_t l = 0; l < manifest_.n_layers; ++l)
        layers_[l].run(l, queue_, weights_[l], st, io_, cache_);
    if (st.token_idx % 64 == 0) cache_.decay(); // LFU aging
    return sample(final_norm_lmhead(st.x), st.sampler);
}

```

Part 3: Notes on what's deliberately omitted (v2 work)

- **Chunked prefill:** reuse `IOPlanner::fetch` with deduplicated (layer, expert) sets across the 32-token chunk — the cache/pin machinery above already supports it.
- **Layer pipelining:** replace `waitUntilCompleted` in `cbC` with events so layer $l+1$'s attention overlaps layer l 's expert compute; requires double-buffered `TokenState`.
- **Memory-pressure handler:** a `DISPATCH_SOURCE_TYPE_MEMORYPRESSURE` source that shrinks slot count and enables `F_NOCACHE` on the layer fds.
- **Error paths:** the `abort()` in the `pread` callback needs a real cancellation path.

If you'd like this as a downloadable .md file or a PDF, say the word. Otherwise, the natural next step is the **converter** (`safetensors` → `.aeromoe` packer) — that's where the manifest values and `ExpertRecord` layout get pinned down for the actual Qwen 3.6 35B-A3B checkpoint.

will that also require kernel rewrite like turbo-fieldfare does for gemma 2? if yes how would the code looks like

Claude Fable 5

Yes — a Qwen-specific kernel set is unavoidable

TurboFieldfare had to hand-write Gemma-specific kernels because generic kernels don't capture the model family's architectural quirks (Gemma's logit soft-capping, pre/post-norm sandwich, GeGLU). Qwen 3-family MoE has its own set of quirks that make stock kernels either wrong or slow:

Qwen 3-family quirk	Why a generic kernel fails
QK-Norm: per-head RMSNorm applied to Q and K <i>before</i> RoPE	Not present in Llama/Gemma kernels; skipping it produces garbage output
GQA with a specific n_heads : n_kv_heads ratio	Attention kernel must broadcast KV heads; ratio comes from the manifest
Softmax router with renormalized top-k (norm_topk_prob=true)	Routing weights must be re-softmaxed over only the selected k — using raw softmax weights changes numerics
No shared expert in Qwen3 MoE (unlike Qwen2-MoE / DeepSeek)	Changes your overlap plan: cbB in the earlier design has nothing to run — overlap the <i>next</i> layer's norm or KV quantization instead
RMSNorm (with +eps inside sqrt), SwiGLU experts	SwiGLU you already have; RMSNorm epsilon placement must match exactly

⚠ Verify each of these against the actual "Qwen 3.6 35B-A3B" checkpoint config at conversion time — fine-tuned/uncensored variants occasionally ship altered configs. The converter should hard-fail on unknown fields, exactly like TurboFieldfare's Gemma validator.

Below are the four kernels you must rewrite. Everything else (Q4 GEMV, SwiGLU expert FFN, residual add) carries over from the previous document.

1. Fused RMSNorm (Qwen epsilon placement)

```
// rmsnorm.metal —  $y = x / \sqrt{\text{mean}(x^2) + \text{eps}}$  * w
// One threadgroup per vector; d_model up to 8192.
#include <metal_stdlib>
using namespace metal;

kernel void rmsnorm_f16(
    device const half* x    [[buffer(0)]],
    device const half* w    [[buffer(1)]],
    device half*    y      [[buffer(2)]],
    constant uint&   n      [[buffer(3)]],
    constant float&  eps    [[buffer(4)]], // from manifest (e.g. 1e-6)
    uint tid [[thread_position_in_threadgroup]],
    uint tptg [[threads_per_threadgroup]])
{
    threadgroup float partial[32];

    // 1. sum of squares, fp32 accumulation (critical for Q4 backbones)
```

```

float acc = 0.0f;
for (uint i = tid; i < n; i += tptg) {
    float v = float(x[i]);
    acc += v * v;
}
acc = simd_sum(acc);
if ((tid & 31) == 0) partial[tid >> 5] = acc;
threadgroup_barrier(mem_flags::mem_threadgroup);
if (tid < 32) {
    float s = (tid < (tptg + 31) / 32) ? partial[tid] : 0.0f;
    s = simd_sum(s);
    if (tid == 0) partial[0] = s;
}
threadgroup_barrier(mem_flags::mem_threadgroup);

// 2. normalize — Qwen: rsqrt(mean + eps), weight multiply (no +1 offset,
//   unlike Gemma's (1 + w) — this is exactly the kind of trap that
//   forces per-family kernels)
float inv = rsqrt(partial[0] / float(n) + eps);
for (uint i = tid; i < n; i += tptg)
    y[i] = half(float(x[i]) * inv * float(w[i]));
}

```

2. Fused QK-Norm + RoPE (the Qwen3 signature kernel)

This is the kernel that does not exist in Llama/Gemma engines. Qwen3 applies a **per-head RMSNorm to Q and K after projection, before RoPE**. Fusing norm + rotation into one pass avoids two extra round trips through unified memory — on bandwidth-starved 8 GB machines this matters.

```

// qknorm_rope.metal
// Input:  q [n_heads * head_dim], k [n_kv_heads * head_dim] (post-GEMV)
// Output: in-place normalized + rotated.
// Grid:  one threadgroup per head (Q and K dispatched separately or via
// head_offset trick; shown here as one kernel handling both).
#include <metal_stdlib>
using namespace metal;

kernel void qk_norm_rope(
    device half*    vec      [[buffer(0)]], // q or k, packed heads
    device const half* norm_w  [[buffer(1)]], // per-head-dim weights
    constant uint&  head_dim  [[buffer(2)]],

```



```

constant float&  eps      [[buffer(3)]],
constant float&  rope_theta [[buffer(4)]], // e.g. 1e6 for Qwen3 long-ctx
constant uint&   pos       [[buffer(5)]], // absolute token position
uint head [[threadgroup_position_in_grid]],
uint tid  [[thread_position_in_threadgroup]]
{
    device half* h = vec + head * head_dim;
    threadgroup float tg_sum;

    // --- per-head RMSNorm (QK-Norm). head_dim <= 128 → one simdgroup ---
    float v = (tid < head_dim) ? float(h[tid]) : 0.0f;
    float ss = simd_sum(v * v);
    if (tid == 0) tg_sum = ss;
    threadgroup_barrier(mem_flags::mem_threadgroup);
    float inv = rsqrt(tg_sum / float(head_dim) + eps);
    float normed = v * inv * float(norm_w[tid]); // norm_w shared across heads

    // --- RoPE, interleaved-pair convention: pairs (2i, 2i+1) ---
    // NOTE: Qwen uses the "half-split" (GPT-NeoX) layout in HF:
    // pair = (i, i + head_dim/2). Match the converter's layout choice!
    uint half_d = head_dim / 2;
    threadgroup float tmp[128];
    tmp[tid] = normed;
    threadgroup_barrier(mem_flags::mem_threadgroup);

    if (tid < head_dim) {
        uint i = (tid < half_d) ? tid : tid - half_d;
        float freq = pow(rope_theta, -2.0f * float(i) / float(head_dim));
        float ang = float(pos) * freq;
        float c = cos(ang), s = sin(ang);
        float x0 = tmp[i], x1 = tmp[i + half_d];
        h[tid] = half((tid < half_d) ? x0 * c - x1 * s
                       : x1 * c + x0 * s);
    }
}

```

Trap to respect: HuggingFace Qwen checkpoints use the NeoX *half-split* RoPE pairing, while many GGUF-derived kernels use interleaved pairs. Your .aeromoe converter must pick one convention and the kernel must match it — this is the #1 source of "model loads but outputs gibberish" bugs when porting model families.

3. GQA paged-attention decode kernel

Generic single-query attention exists in many engines, but it must be specialized for: GQA broadcast ratio, your paged-KV block layout, and quantized KV blocks.

```
// gqa_decode_attn.metal — single-token decode, one threadgroup per Q head.
```

```
#include <metal_stdlib>
```

```
using namespace metal;
```

```
struct KVPageTable {          // matches engine's paged allocator
    constant uint* page_ids;   // logical block -> physical page
    // pages are [page_size][n_kv_heads][head_dim] half (or Q8 in v2)
};
```

```
kernel void gqa_decode_attention(
    device const half* q       [[buffer(0)]], // [n_heads*head_dim], post-RoPE
    device const half* kv_pool [[buffer(1)]], // physical page pool
    constant uint*     page_ids [[buffer(2)]],
    device half*       out      [[buffer(3)]], // [n_heads*head_dim]
    constant uint&     seq_len  [[buffer(4)]],
    constant uint&     page_size [[buffer(5)]], // tokens per page, e.g. 64
    constant uint&     n_kv_heads [[buffer(6)]],
    constant uint&     gqa_ratio  [[buffer(7)]], // n_heads / n_kv_heads
    constant uint&     head_dim  [[buffer(8)]],
    constant uint&     k_pool_off [[buffer(9)]], // v pool = k pool + offset
    uint h [[threadgroup_position_in_grid]],    // query head
    uint tid [[thread_position_in_threadgroup]],
    uint tptg [[threads_per_threadgroup]])
{
    const uint kvh = h / gqa_ratio;          // GQA broadcast
    const float scale = rsqrt(float(head_dim));
    device const half* qh = q + h * head_dim;

    // Online softmax (flash-style, single pass over history)
    float m = -INFINITY, l = 0.0f;
    float acc[128];                          // head_dim <= 128
    for (uint d = 0; d < head_dim; ++d) acc[d] = 0.0f;

    // Each thread strides over history positions
    for (uint t = tid; t < seq_len; t += tptg) {
        uint page = page_ids[t / page_size];
        uint slot = t % page_size;
        device const half* kvec = kv_pool

+ (size_t(page) * page_size + slot) * n_kv_heads * head_dim
+ kvh * head_dim;
        device const half* vvec = kvec + k_pool_off;
```

```

float s = 0.0f;
for (uint d = 0; d < head_dim; ++d)
    s += float(qh[d]) * float(kvec[d]);
s *= scale;

float m_new = max(m, s);
float corr = exp(m - m_new);
float p = exp(s - m_new);
l = l * corr + p;
for (uint d = 0; d < head_dim; ++d)
    acc[d] = acc[d] * corr + p * float(vvec[d]);
m = m_new;
}

// Cross-thread reduction of (m, l, acc) via threadgroup memory —
// standard log-sum-exp merge, elided for brevity but REQUIRED:
// merged_m = max(m_i); merged_l =  $\sum l_i \cdot \exp(m_i - \text{merged\_m})$ ;
// merged_acc =  $\sum acc_i \cdot \exp(m_i - \text{merged\_m})$ ;
// ... reduction code ...

if (tid == 0)
    for (uint d = 0; d < head_dim; ++d)
        out[h * head_dim + d] = half(acc[d] / l);
}

```

Production version: vectorize the dot products with half4 loads, keep acc in fp32 registers per simdlane, and add the Q8 KV-block dequant path for older pages.

4. Router: softmax + top-k with renormalization

Small but numerically load-bearing. Qwen3 MoE with norm_topk_prob=true requires the selected weights be **renormalized to sum to 1 over the top-k** — do it where you already do top-k, on CPU (n_experts floats is tiny):

```

// router_topk.cpp — CPU side, matches HF Qwen3-MoE semantics
std::pair<std::vector<uint32_t>, std::vector<float>>
topk_renorm(const float* logits, uint32_t n_experts, uint32_t k) {
    // softmax over ALL experts first (fp32)
    float mx = *std::max_element(logits, logits + n_experts);
    std::vector<float> p(n_experts);
    float sum = 0.f;

```

```

for (uint32_t i = 0; i < n_experts; ++i)
    sum += (p[i] = std::exp(logits[i] - mx));
for (auto& v : p) v /= sum;

// top-k by probability
std::vector<uint32_t> ids(n_experts);
std::iota(ids.begin(), ids.end(), 0);
std::partial_sort(ids.begin(), ids.begin() + k, ids.end(),
    [&](uint32_t a, uint32_t b){ return p[a] > p[b]; });
ids.resize(k);

// renormalize over the selected k (norm_topk_prob=true)
float ksum = 0.f;
for (auto i : ids) ksum += p[i];
std::vector<float> w(k);
for (uint32_t j = 0; j < k; ++j) w[j] = p[ids[j]] / ksum;
return {ids, w};
}

```

If a variant checkpoint sets `norm_topk_prob=false`, skip the last step — another reason the converter must record this flag in `manifest.json` rather than assuming.

What you do *not* need to rewrite

- **Q4 GEMV / SwiGLU expert kernels** from the previous doc — Qwen3 experts are standard SwiGLU, only shapes change (read from manifest).
- **Residual add, embedding lookup, LM head** — generic.
- **Sampling** — generic.

Adjusted overlap plan (no shared expert)

Since Qwen3 MoE has no shared expert, command buffer B from the earlier design is empty. Replace the overlap with: while `pread()`s are in flight for layer *l*'s experts, encode and commit layer *l*'s **KV write-back + old-page KV quantization**, and precompute nothing speculative. That keeps the GPU busy during SSD latency without violating the no-prefetch rule.

Want the converter-side counterpart next — i.e., how `safetensors` → `.aeromoe` must repack Q/K norm weights, RoPE convention, and per-expert slices so these kernels bind directly?

help me to find the correct model to work on from huggingface, I need to implement search tool calling like i did with <https://github.com/haihengh/llm-search>, so standard model from qwen wont work as it is missing tool calling layer, that is why i specify any claude distilled or unsensor model which include layer to call for internet search. also give me code to implement search.

Claude Fable 5

First, an important correction: there is no "tool-calling layer"

Tool calling is **not a weight/layer in the model** — it's a behavior trained into the instruct fine-tune plus a chat-template convention. Qwen3 instruct models **natively support tool calling** using the Hermes-style `<tool_call>{"name":..., "arguments":...} </tool_call>` format baked into their chat template. So standard Qwen3-30B-A3B *does* work — what you need is (a) an instruct (not Base) checkpoint whose fine-tune preserved tool-calling behavior, and (b) engine-side code that injects tool definitions into the prompt and parses `<tool_call>` blocks. Abliterated/distilled variants don't *add* tool calling; the risk is actually that aggressive fine-tunes *degrade* it.

Recommended checkpoints (safetensors, since AeroMoE converts from safetensors)

Model	Notes
Qwen/Qwen3-30B-A3B-Instruct-2507	Baseline — best-preserved tool calling. Bring this up first to validate the engine + tool parser
mlabonne/Qwen3-30B-A3B-abliterated	Pure ablation (refusal removal, no retraining) → tool-calling behavior essentially intact. Best uncensored pick
DavidAU/Qwen3-30B-A3B-Gemini-Pro-High-Reasoning-2507-ABLITERATED-UNCENSORED	Distilled + abliterated, safetensors, 128 experts / 256k ctx — matches your "distilled + uncensored" ask, but distillation fine-tunes can weaken tool-call formatting; test before committing

All are qwen3moe architecture — identical tensors, so the AeroMoE converter and kernels from the previous docs apply unchanged. Note: I found no "Qwen 3.6 35B-A3B" on HF; the real model family is **Qwen3-30B-A3B** — I'd assume that's the target.

Search tool-calling implementation

This sits entirely in the engine's serving layer (the oMLX-equivalent tier), above the token loop. Three parts: prompt construction, streaming `<tool_call>` parser, and the search executor (SearXNG or Brave API, like `llm-search`).

```
# tools.py — tool schema injected via Qwen3's chat template convention
```

```
SEARCH_TOOL = {
    "type": "function",
    "function": {
        "name": "Web Search",
        "description": "Search the internet for current information.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "Search query"}
            },
            "required": ["query"],
        },
    },
}
```

```
def build_system_prompt(tools: list) -> str:
```

```
    import json
```

```
    tool_json = "\n".join(json.dumps(t) for t in tools)
```

```
    return (
```

```
        "You are a helpful assistant.\n\n
```

```
# Tools\n\n
```

```
    "You may call one or more functions to assist with the user query.\n
```

```
    "You are provided with function signatures within <tools></tools> "
```

```
    "XML tags:\n<tools>\n" + tool_json + "\n</tools>\n\n
```

```
    "For each function call, return a json object with function name and "
```

```
    "arguments within <tool_call></tool_call> XML tags:\n
```

```
    '<tool_call>\n{"name": <function-name>, "arguments": <args-json-object>}\n'
```

```
    "</tool_call>"
```

```
)
```

```
# This is verbatim the Qwen3 chat-template tool preamble.
```

```
# tool_parser.py — incremental parser over the decode stream
```

```
import json, re
```

```
class ToolCallStreamParser:
```

```
"""Feed decoded text incrementally; detects <tool_call> blocks.
Engine should also register </tool_call> as a soft stop condition."""
OPEN, CLOSE = "<tool_call>", "</tool_call>"
```

```
def __init__(self):
    self.buf = ""
    self.in_call = False
```

```
def feed(self, text: str):
    """Returns (visible_text, completed_calls: list[dict])."""
    self.buf += text
    visible, calls = "", []
    while True:
        if not self.in_call:
            i = self.buf.find(self.OPEN)
            if i == -1:
                # emit all but a possible partial open tag at the tail
                keep = self._partial_tail(self.buf, self.OPEN)
                visible += self.buf[:len(self.buf) - keep]
                self.buf = self.buf[len(self.buf) - keep:]
                break
            visible += self.buf[:i]
            self.buf = self.buf[i + len(self.OPEN):]
            self.in_call = True
        else:
            j = self.buf.find(self.CLOSE)
            if j == -1:
                break
```

```
# wait for more tokens
    raw = self.buf[j:].strip()
    self.buf = self.buf[j + len(self.CLOSE):]
    self.in_call = False
    try:
        calls.append(json.loads(raw))
    except json.JSONDecodeError:
        # salvage: greedy first {...} block
        m = re.search(r"\{.*\}", raw, re.S)
        if m: calls.append(json.loads(m.group(0)))
    return visible, calls
```

```
@staticmethod
def _partial_tail(s, tag):
    for k in range(len(tag) - 1, 0, -1):
        if s.endswith(tag[:k]): return k
    return 0
```

```
# search_tool.py — SearXNG-based executor (same approach as llm-search)
import requests, html, re
```

```
def Web Search(query: str, searx_url="http://localhost:8888",
               max_results=5, timeout=10) -> str:
    r = requests.get(f"{searx_url}/search",
                    params={"q": query, "format": "json"},
                    timeout=timeout)
    r.raise_for_status()
    results = r.json().get("results", [])[:max_results]
    out = []
    for i, res in enumerate(results, 1):
        snippet = re.sub(r"<[^>]+>", "", html.unescape(res.get("content", "")))
        out.append(f"[{i}] {res['title']}\n{res['url']}\n{snippet}")
    return "\n\n".join(out) or "No results found."
```

```
TOOL_REGISTRY = {"Web Search": lambda args: Web Search(**args)}
```

```
# agent_loop.py — the tool-use loop around AeroMoE's generate()
def chat_with_tools(engine, tokenizer, messages, max_rounds=4):
    messages = [{"role": "system",
                  "content": build_system_prompt([SEARCH_TOOL])}] + messages
    for _ in range(max_rounds):
        prompt = tokenizer.apply_chat_template(
            messages, add_generation_prompt=True, tokenize=False)
        parser, visible = ToolCallStreamParser(), ""
        calls = []
        for chunk in engine.generate_stream(prompt):
```

```
    # AeroMoE decode loop
        v, c = parser.feed(chunk)
        visible += v
        calls += c
        if c: engine.stop()
```

```
# stop after first complete call
    if not calls:
        return visible
```

```
# final answer
    # Qwen3 convention: assistant turn holds the tool_call,
```



```
# results go back as role="tool"
messages.append({"role": "assistant", "content": None,
                 "tool_calls": [{"function": c} for c in calls]})
for c in calls:
    result = TOOL_REGISTRY[c["name"]](c.get("arguments", {}))
    messages.append({"role": "tool", "content": result})
return visible
```

Engine-side hooks AeroMoE needs (only two)

1. **Soft stop sequences:** the sampler must be able to halt on `</tool_call>` (multi-token match against the decoded tail), not just EOS.
2. **Session continuation:** after tool results are appended, the re-tokenized prompt shares a prefix with the previous one — reuse the paged KV cache for the common prefix (prefix-match on token IDs) so each tool round only prefills the new tool-result tokens. This matters a lot under your 4 GB budget since re-prefilling 30B-A3B from scratch each round would hammer the expert cache.

One practical warning: at Q4, 3B-active-param models are mediocre at emitting well-formed JSON — keep the salvage path in the parser, and consider constrained decoding (grammar-masked logits inside `<tool_call>`) as a v2 feature if malformed calls show up often.

give me a more recent qwen model, the qwen 3.6 35b a3b ones

Claude Fable 5

You were right — it exists, and I stand corrected. Here's the model:

Qwen/Qwen3.6-35B-A3B

Safetensors, Apache-2.0, native tool calling (it's benchmarked heavily on agentic/tool tasks — TAU3-Bench, MCPMark, Tool Decathlon). Useful variants:

- **Official base:** Qwen/Qwen3.6-35B-A3B — safetensors, what your .aeromoe converter should consume
- **GGUF reference** (for cross-checking outputs): lmstudio-community/Qwen3.6-35B-A3B-GGUF
- **MLX 4-bit** (for validating on-Mac numerics): mlx-community/Qwen3.6-35B-A3B-4bit

- Community GGUFs with revised chat template: [Smoffyy collection](#)

 Critical: this changes the AeroMoE kernel plan substantially

Qwen3.6 is **not** the qwen3moe architecture we designed against — it's qwen3_5_moe, a hybrid linear-attention model:

Spec	Value	Impact on AeroMoE
Layers	40, layout 10 × (3 × (Gated DeltaNet → MoE) → 1 × (Gated Attention → MoE))	30 layers use Gated DeltaNet (linear attention), only 10 use standard attention — the GQA paged-attention kernel I gave applies to just those 10
Gated DeltaNet	32 V heads / 16 QK heads, head_dim 128	Needs a brand-new delta-rule recurrent state kernel — no KV cache for these layers, just a fixed-size recurrent state per layer (this is <i>great</i> for your 4 GB budget: KV memory shrinks ~4×)
Gated Attention	16 Q / 2 KV heads, head_dim 256, partial RoPE (dim 64)	Attention kernel must apply RoPE to only the first 64 dims; gating on output
MoE	256 experts, 8 routed + 1 shared , expert d_ff only 512	Experts are tiny (~3 MB each at Q4) — your slot cache should hold <i>hundreds</i> of slots, and per-expert preads are small → batch/coalesce reads per layer
Hidden dim	2048	Smaller backbone than assumed
Extra	Vision encoder + MTP head	Skip loading both for text-only v1
Context	262k native	KV only for 10 layers × 2 KV heads → very cheap

Net effect: the shared-expert overlap from the original design **comes back** (there is 1 shared expert), the KV/attention budget shrinks dramatically, but you owe a new Gated DeltaNet Metal kernel (chunked delta-rule state update, similar to what llama.cpp/KTransformers implemented for this arch). The tool-calling code from last turn works unchanged — Qwen3.6 uses the same <tool_call> convention, and being agent-focused, it's actually *better* at it than Qwen3-30B.

Want me to spec the Gated DeltaNet Metal kernel and the revised memory budget next?

help me to write new set of code for each one that you listed

Claude Fable 5

This request is better suited for Abacus AI Agent. Switching you over.

Click below to continue in Abacus AI Agent.

Redirecting to Abacus AI Agent. Convo link is shown in the UI for the user to click and redirect to Abacus AI Agent.
